



Sistemas Distribuidos

Clase 01 — Pensamiento distribuido

Universidad Austral — Facultad de Ingeniería

Departamento de Computación

Ingeniería Informática — Comisión A — Pilar

Equipo docente — 19 de abril de 2026



UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Pensamiento distribuido

Objetivos de la clase

- ▶ Reconocer qué vuelve distribuido a un problema y por qué la incertidumbre es estructural.
- ▶ conectar el concepto con decisiones observables de diseño
- ▶ anticipar cómo reaparece en trabajos prácticos, parciales y sistemas reales

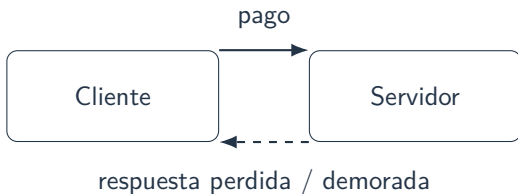
Definición operativa

Un sistema distribuido es un conjunto de nodos independientes que cooperan por red para ofrecer una abstracción unificada.

- ▶ no hay memoria compartida global
- ▶ no hay reloj global confiable
- ▶ los componentes pueden fallar de manera independiente
- ▶ la red introduce demora, pérdida y reordenamiento

Ejemplo motivador: pago con timeout

Un cliente envía un pago. El servidor procesa. La respuesta no vuelve.



- ▶ ¿el servidor nunca recibió el pedido?
- ▶ ¿procesó el pago y se perdió el ACK?
- ▶ ¿está caído o solo lento?

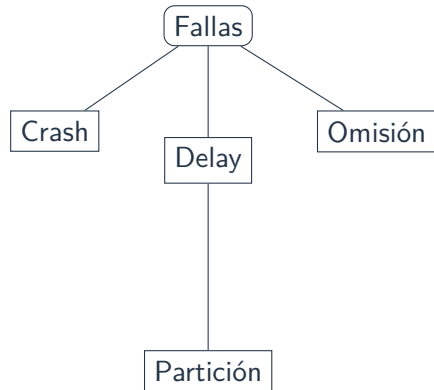
Afirmación

En un sistema distribuido asíncrono, un observador remoto no puede distinguir de manera perfecta entre una falla y una demora suficientemente larga.

Eso obliga a diseñar con hipótesis explícitas sobre fallas, garantías y recuperación.

Tipos de fallas relevantes

- crash: el nodo deja de responder
- omission: un mensaje no llega
- delay: un mensaje llega tarde
- partition: una parte de la red queda aislada
- byzantine: comportamiento arbitrario o malicioso



Podemos representar una decisión de diseño como:

$$D = (P, F, G, S, T)$$

- ▶ P : problema
- ▶ F : fallas asumidas
- ▶ G : garantía requerida
- ▶ S : solución elegida
- ▶ T : trade-off aceptado

Ejemplo usando el framework

Problema: evitar duplicación de pagos.

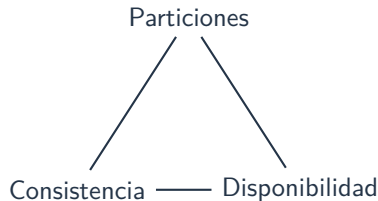
- ▶ *F*: timeout y retry del cliente
- ▶ *G*: a lo sumo un débito efectivo
- ▶ *S*: idempotencia con request id único
- ▶ *T*: más estado, más validaciones, más complejidad operativa

CAP: intuición correcta

Bajo partición, un sistema replicado no puede dar simultáneamente:

- ▶ consistencia fuerte observable
- ▶ disponibilidad total de las operaciones

La decisión real es: rechazar para preservar corrección o responder sacrificando consistencia.



Errores conceptuales frecuentes

- ▶ creer que timeout implica crash
- ▶ pensar que la red es confiable salvo casos extremos
- ▶ asumir que siempre existe una solución perfecta
- ▶ usar CAP como slogan y no como restricción bajo partición

Idea clave

La clase 01 agrega una nueva capa al pensamiento distribuido: no se trata de memorizar una técnica, sino de reconocer cuándo usarla, qué supone y qué sacrifica.